GraphFI: An Efficient Fault Injection Framework for Graph Processing on GPGPUs

Nan Jiang*, Hengshan Yue*, Jingweijia Tan*, Mengting Zhou*, Xiaonan Wang*,
Yuchun Wang*, Wenda Wei*, Meikang Qiu†, and Xiaohui Wei*
*College of Computer Science and Technology, Jilin University
†School of Computer and Cyber Science, Augusta University

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11133241

***Abstract*—As graph tasks become pervasive in real-time and safety-critical domains (e.g., financial fraud detection and electrical power systems), it is also essential to guarantee their reliable execution beyond pursuing extraordinary performance. However, due to the neglect of consideration for graph-specific execution paradigm, existing Fault Injection (FI) reliability analysis methods typically incur inaccurate system error resilience characterization, making it challenging to provide helpful guidance for efficient and reliable graph processing paradigm design.**

This paper proposes *GraphFI*, an efficient *Graph Fault Injection* framework on the universal parallel tasks acceleration platform (i.e., GPGPUs). Our key insight is progressively excavating the graph-specific error propagation and effect mechanisms, thereby avoiding blind FI trials. Firstly, observing that iterations with similar active vertex set exhibit similar error behavior, we propose *i*teration-*d*riven *GraphFI* (*ID-GraphFI*) to solely select representative iterations for fast error resilience profile assessment. Secondly, by detecting resilience-similarity communities in graph topology, we propose *t*opology-*d*riven *GraphFI* (*TD-GraphFI*) that only selects representative vertices for community overall reliability evaluation. Thirdly, by exploring the graph-specific fault monotonic property, we propose the *m*onotonicity-*d*riven *GraphFI* (*MD-GraphFI*) to granularly draw system severe error boundaries for predictable/unnecessary fault injection avoidance. Merging them all, GraphFI can reduce system fault site space by up to two orders of magnitude, which achieves 2.1∼15.2× speedup compared to SOTA methods while providing better reliability assessment accuracy.

I. Introduction

As a flexible data structure for representing complex relationships between objects, graphs are widely utilized in important real-world scenarios. Due to the inherent parallelism of graph processing, General-Purpose GPUs (i.e., GPGPUs) have established their prevalence in accelerating graph processing [1]–[4]. While delivering performance improvement, the increasing hardware fault rate (e.g., transient bit-flip faults [5], [6]) of highly integrated GPGPU platforms bring severe reliability challenges to graph processing. Even worse, the extensive graph data interaction enables abundant fault propagation, further exacerbating this challenge. The hardware imperfections may cause graph system to suffer from *obvious system crashes* or *silent but severe output corruptions*. As graph processing becomes pervasive in safety-critical real-world domains (e.g., financial fraud detection and electrical power system), the above severe fault-incurred consequences may lead to non-negligible system state recovery overhead [7] or financial loss/penalty [8], [9]. As a result, beyond pursuing extraordinary performance, it is also essential to guarantee their reliable execution.

To design an efficient and reliable graph processing paradigm, the first and vital step is to systematically analyze the reliability profile of the graph processing system. Fault Injection (FI) is the classic method which can identify the system's fault-sensitive area by randomly inserting errors during program runtime and aggregating the corrupted results. Recently, multiple FI-analysis tools have been developed by both industries (e.g., SASSIFI [5] and NvbitFI [10] proposed by NVIDIA) and academia [6], [11], [12]. However, abundant FI trials are necessary to obtain statistically significant analytical conclusions, which typically incur a non-negligible time overhead in GPGPU as each FI trial indicates a complete program execution.

To bridge the inefficiency of traditional FI, a limited set of studies aim to accelerate the FI-based methodology by exploring the program fault impact similarity/monotonicity property [13]–[18]. For example, Hari et al. [13] observe threads with similar dynamic instruction sequences (i.e., similar execution patterns) show similar error behavior, thus only conducting reliability assessment for representative threads. Observing more significant fault-incurred deviations typically produce more severe output corruptions, Li et al. [17] preferentially search for the minimum deviation that can cause severe corruptions, thus avoiding predictable FI trials. Although showing superiority for fast system reliability assessment, directly applying theses methods may lead to inaccurate graph reliability evaluation due to the specific data topology and execution paradigm. For instance, the graph topology may decouple the relationship between thread dynamic instruction sequence and their error behavior, and the graph-specific updating rules may also violate the error monotonic property.

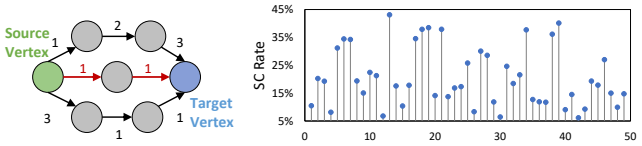
Considering the urgent demand for the reliability and performance co-optimization of graph processing systems, we build *GraphFI*, an efficient *Graph Fault Injection* method on GPGPUs. *The core idea and main novelty of our approach is that we progressively excavate the fault propagation and effect mechanisms hidden in the execution paradigm and data topology of graph processing, thereby guiding targeted and efficient FI trials.* Specifically, observing that iterations with similar active vertex set exhibit similar error behavior, we propose *i*teration-*d*riven *GraphFI* (*ID-GraphFI*) to solely selects representative iterations for fast error resilience profiling. Secondly, by detecting resilience-similarity communities in graph topology, we propose *t*opology-*d*riven *GraphFI* (*TD-GraphFI*) that only select representative vertices for community overall reliability evaluation. Thirdly, by exploring the graph-specific fault monotonic property, we propose the *m*onotonicity-*d*riven *GraphFI* (*MD-GraphFI*) to draw error boundaries for predictable/unnecessary fault site injection avoidance. Merging them all, GraphFI can reduce the fault site space of graph programs for up to two orders of magnitude and achieve 2.1∼15.2× speedup compared to SOTA methods while providing better reliability assessment accuracy.

To the best of our knowledge, GraphFI is the first work that can perform fast reliability assessment for GPGPU-based graph processing. In summary, we make the following contributions:

- Explore the graph-specific error propagation and effect mechanisms and observe the (1) *fault similarity property* among iterations/vertices and the (2) *fault monotonicity property* in graph execution process, which provide opportunities for fault space reduction.
- Propose GraphFI that progressively reduce the fault space to efficiently and accurately evaluate the reliability profile of GPGPU-

This work is supported by the National Key Research and Development Program of China under Grant No. 2023YFB4502304, the National Natural Science Foundation of China (NSFC) under Grants No. 62272190, No. 62302190 and No. 62372207, and the Graduate Innovation Fund of Jilin University under Grant No. 2024CX091.

Corresponding authors: Hengshan Yue (yuehs@jlu.edu.cn) and Xiaohui Wei (weixh@jlu.edu.cn).



(a) (b)

Fig. 1. (a) An example of critical edges in Single Source Shortest Path (SSSP) and (b) The SC rate of 50 vertices which have similar neighbor numbers from Pagerank (PR).

bases graph processing.

- Evaluate the efficiency and accuracy of GraphFI on reliability profile estimating of four popular graph programs. We also demonstrate GraphFI is superior than other SOTA methods.
- Demonstrate how we can leverage GraphFI to guide design fault tolerance strategy GPGPU-based graph processing.

II. Background and Motivation

A. Graph Processing on GPGPUs

Graphs are typically represented in the form of $G = (V, E)$, where V represents the vertices set, and E denotes the edge set that represent the relationships between vertices. The purpose of graph processing is to explore complex vertices relationships by iteratively updating vertices states based on the adjacency relationships. In each iteration, vertices that receive new states in the last iteration are marked as active, which need to conduct the update operation. Due to the specific data topology, graph algorithms can be decomposed into a series of homogeneous vertex/edge tasks, exhibiting inherent compatibility to parallel computation.

Packed with thousands of computing units, GPGPUs have established their prevalence in accelerating parallel tasks. To fully utilizing the parallel potential of GPGPUs, *vertex-centric* [1], [19], [20] and *edge-centric* [21], [22] graph programming model become pervasive in recent years. In vertex-centric model, a thread serves as the computation carrier of individual vertex, including neighbor information aggregation, vertex state update, and state propagation. However, as the workload of each thread is proportional to its neighbor number, the vertex power-law distribution (i.e., a few vertices have numerous neighbors) in graph may lead to thread load imbalance in vertex-centric model. Then, edge-centric model is proposed to mitigate this imbalance. In this model, a thread operates on single edge to atomically update its destination vertex state using the source vertex state, resulting in similar workload among threads. However, compared to vertex-centric model, much more atomic operations are required as multiple threads may simultaneously write to the same vertex, which may become system performance bottleneck.

In above models, all vertices perform state updates and propagate synchronously in each iteration. To pursue a more efficient graph processing paradigm in GPGPU, recent studies [4], [23], [24] propose the asynchronous execution model that allows immediate vertex state propagation by relaxing the strictly synchronous barriers. For example, AsynGraph [4] considers the graph sketch constructed with high-degree vertices typically plays an important role in convergence speed, thus conducting multiple round state updating for the graph sketch in each iteration to accelerate graph convergence[1].

B. Reliability Challenge in Graph Processing

Although delivering performance improvement, the increasing hardware fault rate (e.g., transient bit-flip faults) of highly integrated GPU platforms bring severe challenges to graph processing. Even worse, the extensive graph data interaction enables abundant fault propagation, further exacerbating this challenge. Under fault perturbation, graph processing may suffer from serious behaviors, e.g., *system*

[1]In this paper, we mainly focus on fast reliability assessment under the vertex-centric model. We also discuss the adaptiveness of GraphFI to edge-centric and asynchronous models in Section VI.

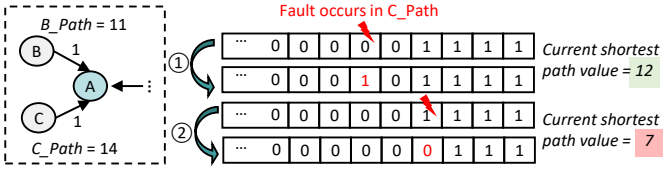


Fig. 2. An example of the un-monotonic fault impact to vertex state updating in Single Source Shortest Path (SSSP).

Detected symptoms (Detected) and *silent but Severe output Corruption (SC)*. Detected cases typically include segmentation faults, fatal traps, application aborts, etc. SC cases indicate graph application produces unacceptable output quality degradation without obvious system anomaly.

Typically, Checkpoint and Recovery (CR) mechanisms [7], [25] can be employed to restore system states when receiving crash signals. However, this may incur non-negligible overhead as heavy memory is required to store abundant immediate graph states. More seriously, CR becomes invalid for silent SCs because there are no obvious system symptoms. Therefore, modular redundancy-based mechanisms [26], [27] (e.g., DMR or TMR) are typically needed for system error detection and correction. However, such redundancy will seriously affect system parallelism, leading to considerable performance degradation. As graph-enabled relevant techniques become pervasive in real-time and safety-critical domains (e.g., financial fraud detection and electrical power systems), it is essential to guarantee reliable graph processing while maintaining computational efficiency. *To this end, the first and vital step is to systematically analyze the reliability bottleneck of graph processing, thereby providing helpful guidance for efficient fault-tolerance strategy design, which is the focus of this study.*

C. Existing Efforts on Program Reliability Analysis

Statistical Fault Injection (SFI) is a traditional approach for program reliability analysis. By randomly inserting *transient bit-flip errors*[2] during program runtime and aggregating the corrupted results, researchers can identify the system's fault-sensitive areas (i.e., reliability bottleneck). Recently, multiple FI-based reliability analysis tools have been developed by both industries (e.g., SASSIFI [5] and NvbitFI [10] proposed by NVIDIA) and academia [6], [11], [12]. However, due to the enormous fault space of GPGPU programs (up to billions [14]), traditional SFI needs abundant trials to achieve statistically significant analytical conclusions, which may incur unacceptable resource- and time-overhead.

Recent efforts attempt to improve the reliability analysis efficiency [13]–[18], [26]. The first category explores program areas that have similar behavior under faults impact, thus only performing FI trials on representative areas to avoid blind FIs. For example, Nie et al. [14] observe threads sharing similar dynamic instruction count often exhibit similar error behavior in GPGPU programs, thus accordingly cluster fault-similar thread groups. FI trials on only one thread are enough to understand the error behavior of the corresponding thread group. More rigorously, Anwer et al. [13] utilize dynamic instruction sequence as the criteria for generating fault-similar thread groups. Observing faults occurring in the operand definition and its first use will result in the same impact (i.e., def-use equivalence pruning), Hari and Pusz et al. [15], [18] spare operand-level FI experiments.

The second category utilizes the fault impact monotonicity property to guide efficient FI methodology [16], [17], [26]. The key insight behind the monotonicity is that larger fault-incurred deviations typically produce more severe output corruptions, and vice versa. For example, Chen et al. [16] notice the fault impact monotonicity in

[2]In this paper, we mainly focus on the transient bit-flip faults in line with previous studies [5], [10], [28].

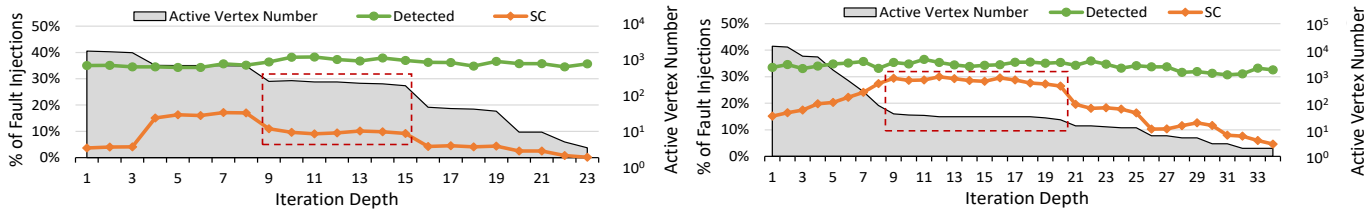


Fig. 3. The relationship between iteration error behavior and its active vertex number of (a) Strongly Connected Component (SCC) and (b) Pagerank (PR).

some AI applications, thus preferentially performing FIs on bits that may incur large deviations to efficiently identify fault-sensitive bits. Also leveraging this monotonicity, in some HPC applications (e.g., fast Fourier transform), Li et al. [17] perform progressively FIs to search the minimum deviation that can cause SCs, thus identifying the fault-tolerant/sensitive area.

D. Motivation

The above efforts show superiority in improving FI efficiency of traditional programs. However, due to the specific data topology and execution paradigm, directly applying these methods to graph applications may lead to inaccurate reliability evaluation. For example, previous studies [13], [14] consider threads with similar dynamic instruction sequences (DIS) to have similar error behavior in GPGPUs. However, this phenomenon does not always exist in GPGPU-based graph processing. For example, all threads have similar DIS in edge-centric model. However, some edges (i.e., threads) may play important roles in graph state updating, thus are more sensitive to faults. Fig. 1(a) illustrates an example of Single Source Shortest Path (SSSP) that uses *min()* to aggregate in-edge vertices' path and calculate the shortest path of each vertex to the source vertex. Obviously, faults occurring on the shortest path (marked with red) are more likely to cause SCs. In vertex-centric model, similar DIS only indicates that vertices have similar neighbor numbers. However, these vertices may have different significance in graph processing (e.g., the influencers of social networks), exhibiting different error sensitivity. Fig. 1(b) shows the SC rate of 50 vertices with similar neighbor numbers in PageRank. The significant fluctuation further reminds us to carefully explore the fault impact similarity existing in graph processing on GPGPUs.

Besides, considering larger fault-incurred deviation typically results in more severe output corruption, previous studies [16], [17] efficiently explore the error boundary that separates the error sensitive and error robust area. However, this monotonic property does not always hold in graph processing due to the specific graph updating rules. For example, as shown in Fig. 2, the current shortest path of A is 12 obtained from B. The fault in low order bit of C_Path will change A's shortest path from 12 to 7. While, fault in high order bit of C_Path will not lead to path corruption, even though the fault introduces larger data deviation. Thus, the specific updating rules of graph processing bring challenges for fault behavior exploration.

In conclusion, it is difficult to efficiently and accurately estimate the reliability of graph processing utilizing existing solutions. In this paper, we first explore the hidden reliability patterns within the specific data topology and execution paradigm of graph processing (Section III). Then we introduce the overall workflow of our proposed GraphFI (Section IV).

III. Error Resilience Characterization for GPGPU-Based Graph Processing

In this section, we progressively excavate the graph error behavior characteristics from the perspective of iterations, data topology, and graph execution to explore fault space reduction opportunities.

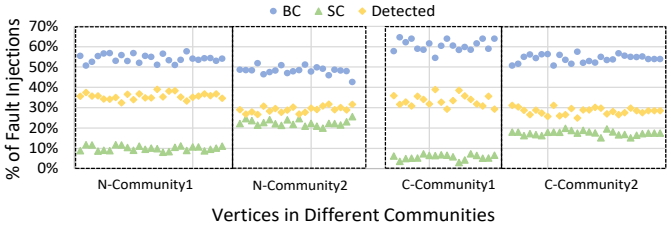


Fig. 4. Resilience similarity among vertices (1) with similar neighborhood (N-Community) and (2) in subgraph with high clustering coefficient (C-Community). BC indicates *Benign Corruption* in the output, which can be accepted by users.

A. Iteration Perspective

Graph processing iteratively updates vertex states until convergence. In an iteration, only active vertices need to conduct the update operation. With the dynamic evolving active vertex set, the error behavior may vary throughout the iterations. Thus, we first focus on the reliability regularity among iterations.

Fig. 3 shows the relationship between iteration error behavior and its active vertex number. We first observe that the Detected trend remains smooth with the changing active vertex number. Detected cases typically arise in state propagation operation (i.e., neighbor state loading). The ratio of loading operation generally remains stable although more active vertices may produce more loading operations. In contrast, the SC rate irregularly fluctuates in investigated benchmarks. Thus, additional effort is required for fast SC proneness estimation. Fortunately, we notice that iterations with similar active vertex sets demonstrate similar SC rates (e.g., iteration 9-15 and iteration 9-20 in the dashed boxes in Fig. 3(a) and Fig. 3(b)). Similar active vertex sets indicate similar data topologies that participate in computation, thus resulting in similar fault propagation and impact behavior. Finally, for iterations with obviously varying active vertex sets, there is no obvious criteria for fault similarity judgment. Thorough FI is suggested to ensure the accuracy of reliability analysis.

Heuristic#1*: We first suggest to construct iterations with similar neighbor loading operation ratio into a Detected-case similarity group. As the loading operation ratio generally stays stable, only individual iteration is enough for overall Detected rate estimation. Then, the similar SC rate of iterations with similar active motivates us to accordingly generate SC-case similarity groups, where one iteration can represent the SC proneness of the whole group.*

B. Topology Perspective

In graph processing, vertices conduct massive state interaction, which provide plentiful opportunities for fault propagation. Thus, we attempt to explore opportunities for efficient reliability analysis from data topology perspective.

A reasonable hypothesis is that vertices with similar neighbors share similar error manifestation, as they exhibit similar fault propagation behavior. Fig. 4 verifies this assumption by illustrating the fault result distribution of vertices with similar neighborhoods (N-Community). We use the Jaccard Coefficient as the proxy of vertex neighborhood similarity which is defined as:

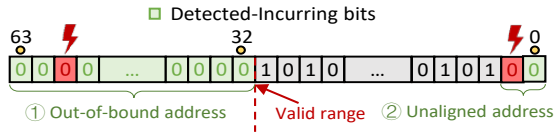


Fig. 5. An example of address operand.

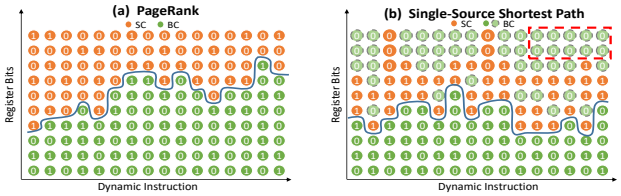


Fig. 6. The error behavior and the error boundaries in (a) PageRank (PR) and (b) Single Source Shortest Path (SSSP).

$$JC\left(v_i, v_j\right) = \frac{\left|N\left(v_i\right) \cap N\left(v_j\right)\right|}{\left|N\left(v_i\right) \cup N\left(v_j\right)\right|} \tag{1}$$

where v_i and v_j represent two vertices with the neighbor sets of $N\left(v_i\right)$ and $N\left(v_j\right)$. Beyond strictly similar neighbors, we observe that vertices in highly clustered groups also exhibit similar error behavior due to their frequent intra-group fault propagation. The C-Communities also illustrate the similar vertices fault distribution.

Heuristic#2*: Utilizing the correlation between vertices' error resilience and their neighborhood-similarity/clustering-property, we identify fault equivalent vertex communities for fast reliability assessment.*

C. Execution Perspective

In this section, we attempt to improve the Monotonicity-guided method by profiling the graph execution to help speed up graph error resilience analysis.

1) Detected Cases: As mentioned in Section III-A, almost all Detected cases originate from the neighbor state loading operation. Fig. 5 demonstrates an example of an address operator, which is typically 64-bits. Actually, we can directly foresee the Detected-incurring bits. Firstly, as the system will allocate a valid address range to the program, errors beyond a certain high order bit of the address will generate an out-of-bound memory. Beyond this, other Detected cases originate from the errors in the lower 2 bits, which will cause unaligned memory accesses.

Heuristic#3*: By checking the valid address range allocated to the program and identifying the Detected-incurring bits, we can directly estimate the Detected rate of the program without FI.*

2) SC Cases: In contrast to Detected cases, SC cases are typically caused by neighbor information aggregating and vertex state updating. Fig. 6(a) illustrates the FI results of Pagerank, where the x-axis represents the instruction sequence and the y-axis represents fault sites (i.e., different bits of operand). We observe larger fault-incurred deviation (i.e., higher bits) are more likely to result in SCs and there is generally a clear SC-BC boundary (blue line in Fig. 6(a)). This motivates us to preferentially search fault boundary to spare FI experiments.

However, we observe complicated error distribution of some graph applications. As shown in Fig. 6(b), several BC cases appear in operand higher bits (the dashed circles). We observe these "abnormal" BCs can be attributed to the specific updating rules of graph algorithm[3]. For example, SSSP uses *min()* to select the shortest path of all in-edge neighbors and accordingly update the target vertex path. Faults in higher bits of non-shortest path operands (e.g., the bits in the dashed box) may be eliminated by the *min()* function (e.g., the example in Fig. 2 in Section II-D). Apart from these special bits, the previously mentioned "boundary" is still observed in the remaining bits. Thus, excluding the abnormal bits, we can also perform fast reliability assessment by finding the fault boundary.

Heuristic#4*: Based on the specific graph updating rules, we can first determine the operand bits whose fault behaviors obey the monotonicity. Then a fast reliability assessment can be achieved by preferentially searching the fault boundary.*

[3]This phenomenon appears widely in many graph applications, such as the *min()/max()* in SSSP, SCC, WCC, MST, SSWP, etc.

IV. GraphFI Architecture

In this section, we first explain the high-level design of GraphFI, then present the details of the implementation steps, and finally show how to utilize GraphFI to help design fault tolerance strategies.

A. Design Overview

Guided by the heuristics proposed in Section III, GraphFI conducts progressive fault site reduction to improve FI efficiency. Fig. 7 depicts the workflow of GraphFI. GraphFI first performs program profiling to obtain (1) *Golden output* (i.e., the fault-free output), which can be utilized to judge whether FIs introduce SCs under the user-defined *Quality metrics*, (2) *Active vertex set of each iteration* and *Neighbor loading operation ratio*, (3) *Graph topology*, and (4) *Valid address profile and Updating operation* as the input of iteration-driven GraphFI (ID-GraphFI), topology-driven GraphFI (TD-GraphFI), and monotonicity-driven GraphFI (MD-GraphFI), respectively. In ID-GraphFI, we identify the fault similarity iteration groups and select one iteration in each group for fast reliability assessment. In each selected iteration, TD-GraphFI identifies fault equivalent vertex communities and also chooses one vertex from each community for community resilience estimation. In each selected vertex, MD-GraphFI draws error boundaries for predictable/unnecessary FI avoidance. Finally, GraphFI accumulates the FI results to obtain the resilience profile of different granularities of graph processing on GPGPUs.

B. Design Details

1) ID-GraphFI: Based on *Heuristic#1*, we first check the neighbor state loading operation ratio of different iterations. As this ratio typically remains stable, we randomly select individual iteration to represent the overall Detected rate of the program. Then we divide consecutive iterations with similar active vertex sets into an iteration group, and select a representative iteration from each group as FI candidate to efficiently evaluate the program SC rate.

2) TD-GraphFI: Within each selected iteration, based on *Heuristic#2*, GraphFI identifies vertex communities in which vertices (a) share similar neighbors (N-Communities) or (b) in highly clustered subgraphs (C-Communities). To identify N-Community, we first screen out vertex pairs with high Jaccard Coefficient (JC). Then a depth-first traverse is conducted towards these pairs to generate communities where vertices have pairwise similar neighbors. For C-Community, we calculate each vertex's Clustering Coefficient (CC). Vertices whose CC surpass the threshold are then gathered into distinct communities based on their connectivity. Then we select one vertex from each community as FI candidate. Vertices belonging to multiple communities are preferentially selected for maximizing FI efficiency improvement.

3) MD-GraphFI: We record the dynamic instruction instances (including the operand values) of each thread as the whole fault space. Within each selected vertex (i.e., thread), based on *Heuristic#3*, we first check the dynamic memory profile and accordingly determine the Detected-incurring bits [15]. Then we calculate the ratio of Detected-incurring bits, which is actually the Detected rate of the program. Secondly, based on *Heuristic#4*, we first exclude the "abnormal" bits whose error-incurred outcome is surely BC according to the fault masking property of the graph updating operation (if there is). Then, a

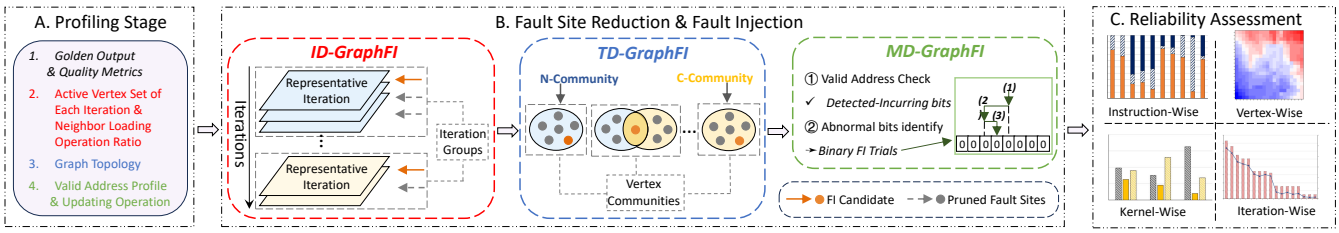


Fig. 7. Overall workflow of GraphFI.

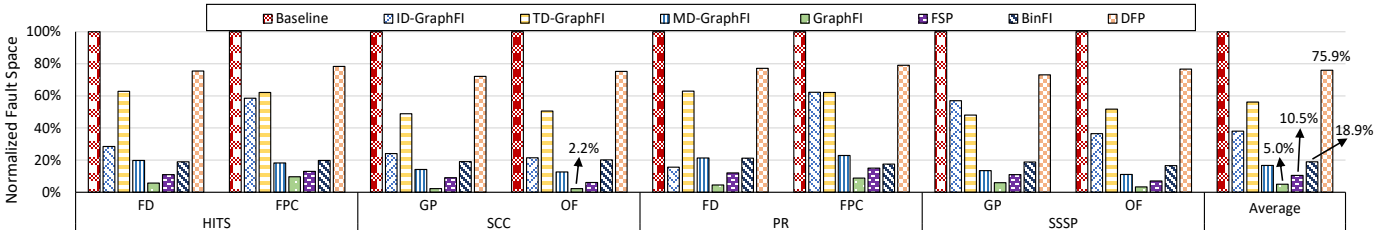


Fig. 8. The speedup of GraphFI and SOTA methods compared to baseline FI, where *ID-GraphFI*, *TD-GraphFI*, and *MD-GraphFI* represent only corresponding mechanism is adopted.

binary search FI is conducted to efficiently identify the fault boundary thus obtaining the instruction level SC rate. Specifically, if the FI result of a bit is SC, then the next FI trial moves to the lower bits until a certain bit is found, which separates the SC and BC cases.

4) Error Resilience Estimation and Guidance for Fault Tolerance Design: After conducting progressive fault site pruning and binary FI experiment, we obtain the instruction-wise error behavior. On this basis, we can accumulate the instruction-level FI results to obtain the error resilience of other system granularity (e.g., thread, and iteration level). For example, the SC rate of individual thread (i.e., vertex) in an iteration can be calculated by:

$$R_{SC} = \sum_{i \subset I_V} A(i) \tag{2}$$

where $A(i)$ is the SC rate of individual instruction and I_V is the instruction set in the target thread. Similarly, iteration-wise SC rate is the accumulation of its thread set.

Next we introduce how to leverage the analysis results of GraphFI to help design efficient fault tolerance strategies for GPGPU-based graph processing. Among the three outcome categories of graph processing (i.e., BC, SC, and Detected), SC is typically the most challenging to detect due to its silent property. Selective thread duplication (ThdDup) and instruction duplication (InsDup) are classical mechanisms to detect silent SCs in GPGPUs by double executing SC-Prone threads/instructions and comparing their results at runtime. We can use GraphFI to efficiently obtain instruction- and thread-level SC rates. Then, GraphFI preferentially duplicates threads/instructions with high SC proneness to pursue efficient SC detection capability.

V. Evaluation

We evaluate the proposed GraphFI by performing the following research questions (RQs):

- RQ_1 What is the performance of GraphFI in evaluating reliability of GPGPU-based graph processing?
- RQ_2 What is the accuracy of GraphFI in evaluating reliability of GPGPU-based graph processing?
- RQ_3 What is the performance of GraphFI-guided fault tolerance mechanism?

A. Experimental Setup

GraphFI settings. We deploy GraphFI on a host Intel i7-7700 CPU (2.60 GHz) with 16 GB memory and an NVIDIA GeForce RTX 3090 GPU. We use a cycle-accurate, open-source simulator GPGPU-sim [29] to obtain the dynamic instruction instances of each thread

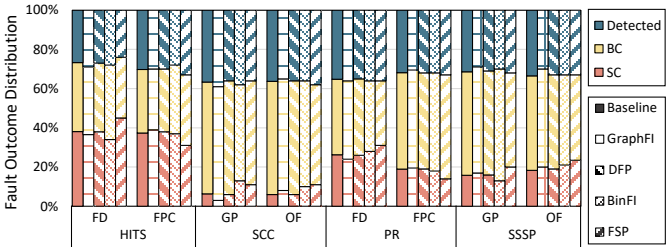


Fig. 9. Error resilience profile comparison of GraphFI and SOTA methods against the baseline FI.

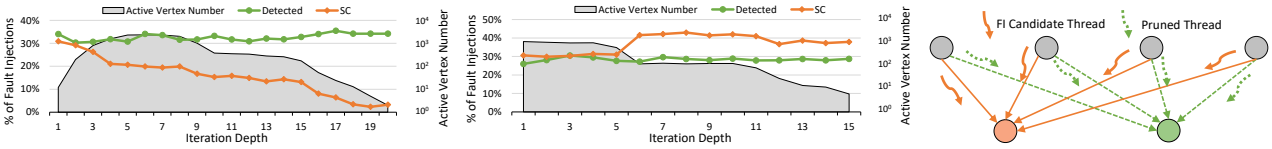
(i.e., the whole fault space). We perform FI trials following the basic FI method in [10], which injects single bit-flip errors into general-purpose registers of dynamic instructions.

Dataset, benchmarks, and output quality metrics. Four popular graph algorithms are considered in our experiment, including *Single Source Shortest Path* (SSSP), *Strongly Connected Component* (SCC), *Pagerank* (PR), and *Hypertext Induced Topic Search* (HITS). We perform evaluations on four real-world graphs (i.e., FD [30], GP [31], FBP [31], and OF [31]). Incorporating the domain-specific knowledge [32], [33] and the reference to previous error resilience studies [34], [35], we adopt *Top-k ranking* for PR and HITS as users typically only care about the correctness of the top-ranked items. For SSSP and SCC, we use *Relative L2-norm* to estimate the deviation of the overall graph's paths/connectivity from the ground truth.

Comparison methods and baseline. We select three SOTA reliability estimation methods for comparison, i.e., (1) FSP [14], (2) DFP [18], and (3) BinFI [16]. FSP clusters thread equivalent groups using dynamic instruction count as criteria and select representative threads for FI. DFP considers instruction semantics and tracks the information flow of each bit to construct fault equivalent bit sets for fault space reduction. Guided by the monotonic property, BinFI binary searches 0 bit and 1 bit separately of each operand to find the SC boundary. All these methods and GraphFI take FI to the whole fault site space as the baseline for comparison. As the faults occurring in the operand definition and its first use will result in same impact (as illustrated in Section II-C), the def-use pruning method has become a popular approach to reduce the fault space. In the following evaluation, we perform a preliminary fault space reduction using the def-use pruning for GraphFI and SOTA solutions.

B. Speedup of GraphFI (RQ_1)

Fig. 8 shows the FI efficiency of all methods by depicting the normalized reduced fault space compared to the baseline. ID-



(a) (b) (c)

Fig. 10. The iteration error behavior of (a) SSSP in edge-centric model and (b) HITS in AsynGraph model. (c) is an example of representative threads selection for edge-centric model.

TABLE I
THE ERROR DETECTION CAPABILITY (I.E., ERROR COVERAGE) OF GRAPHFI- AND FSP-GUIDED INSTRUCTION DUPLICATION (INSDUP) AND THREAD DUPLICATION (THDDUP) METHOD.

Overhead		25%	50%	75%
InsDup Detection	GraphFI	70.3%±2.8%	90.1%±1.2%	95.6%±0.9%
	FSP	65.5%±3.3%	82.9%±2.1%	93.5%±1.2%
ThdDup Detection	GraphFI	80.2%±4.5%	89.9%±2.9%	96.6%±1.8%
	FSP	57.1%±7.2%	77.1%±5.4%	90.6%±2.2%

GraphFI, TD-GraphFI, and MD-GraphFI achieve an average fault space reduction of 62.0%, 43.8%, and 83.3%, respectively. Merging them all, GraphFI can compress the fault space to 5.0% of its original size on average, achieving a speedup of up to two magnitudes to baseline FI. In general, GraphFI exhibits 2.1∼15.2× speedup to other SOTA methods. For FSP, as vertices' degrees exhibit significant variation (i.e., more thread groups), FSP has to select abundant threads to perform FIs. The efficiency of DFP is mainly proportional to the ratio of logical/data-movement instructions (e.g., MOV, SHL, AND, etc.), while these instructions usually occupy a small portion of the program. To ensure accurate reliability evaluation, BinFI needs to respectively perform binary search FI for 0 bits and 1 bits of each operand, which could incur non-negligible overhead (e.g., up to 8 FIs are necessary for a 32-bit operand).

C. Overall Error Resilience Evaluation (RQ_2)

Fig. 9 shows the reliability profile obtained by GraphFI and other SOTA methods. On average, the difference of GraphFI compared to baseline FI in terms of Detected, SC, and BC rates are 0.96%, 1.36%, and 1.22%, respectively. The delicate deviations indicate that GraphFI achieves comparable accuracy with the ground truth when evaluating the error profile of graph processing. Compared to GraphFI, the results of FSP exhibit more obvious discrepancy and volatility to the ground truth. Because the graph data topology decouples the relationship between threads' dynamic instruction count and their error resilience. DFP and BinFI also achieve promising accuracy in estimating error resilience as they adopt relatively conservative principles to reduce fault space. However, this leads to unsatisfactory efficiency due to the lack of consideration of graph-specific characteristics. In general, GraphFI achieves a better tradeoff between resilience evaluation efficiency and accuracy than SOTA solutions.

D. GraphFI-Guided Fault Tolerance (RQ_3)

Table. I shows the SC detection rate at different protection overhead, which means the increasing dynamic instruction or thread ratio for InsDup and ThdDup. In general, we observe that GraphFI-guided duplication exhibits better SC mitigation efficiency and stability compared to FSP, especially for ThdDup. This further verifies the accuracy of GraphFI in terms of graph error resilience estimating in different system granularities.

VI. DISCUSSION: THE EFFECTIVENESS OF GRAPHFI TO OTHER GRAPH PROCESSING PARADIGM

In addition to the synchronous vertex-centric model (VC), we further discuss the adaptiveness of GraphFI to synchronous edge-centric (EC) and asynchronous graph processing models (taking AsynGraph [4] as an example) illustrated in Section II-A.

Iteration perspective. In VC, our key insight is that iterations with similar active vertex sets show similar error behavior. Such a phenomenon is observed in other models. Fig. 10(a) shows the error behavior of different iterations in EC model. Here, similar active vertex sets imply similar thread number and graph topology that participate in computation, also exhibiting similar fault propagation patterns and error behaviors. AsynGraph considers the graph sketch constructed with high-degree vertices play an important role in convergence speed, thus conducting multiple round state updating for this graph sketch in each iteration. In fact, we observe these high-degree vertices (less than 20% of total vertices) contribute over 80% SCs of the program, thus analysis for these vertices can generally understand the program resilience. Fig. 10(b) illustrates the error behavior of the high-degree graph sketch with iterations going on. We observe that high-degree vertices gradually converge with iterations, meanwhile also exhibiting phase-wise similar error resilience.

Vertex perspective. By analyzing the fault propagation and impact, we find the resilience similarity in vertices within highly clustered subgraphs or having similar neighbor. Such similarity also exist in other models because these models generally do not change the fault propagation behavior in data topology. Note that, we should pay more attention to FI candidate thread selection. For example, as shown in Fig. 10(c), a vertex updating task is divide into four threads in EC model. As the orange vertex and the green vertex have the same neighborhood, we can only select all four orange threads as FI candidate.

Execution perspective. Diverse graph processing models on GPUs essentially modify the granularity of task division (e.g., VC and EC models) or the way state updates are propagated (e.g., synchronous and asynchronous models), rather than changing the graph update algorithm. Our approach focuses on exploring fault space reduction opportunities based on graph algorithms and the impact of faults, which remain unaffected with different graph processing models.

In conclusion, the proposed heuristics for fault space reduction are generally appropriate for the popular graph processing models.

VII. CONCLUSION

In this paper, we propose GraphFI, an FI framework that can efficiently and accurately estimate the reliability profile of GPGPU-based graph processing. The main effort is that we reveal the error propagation and effect mechanisms hidden in the execution paradigm and data topology of graph processing on GPGPUs. Specifically, we observe (1) iteration-wise error behavior similarity exists in iterations sharing similar active vertex set, (2) vertex-wise error behavior similarity among vertices with similar neighbors or in highly clustered subgraphs, and (3) instruction-wise graph algorithm-specific fault impact monotonicity property. Incorporating these characteristics, GraphFI progressively prunes the fault space and performs targeted and efficient FI trials. Experimental results show that, in estimating the reliability of GPGPU-based graph processing, GraphFI can achieve a 2.1∼15.2× speedup compared to SOTA methods, while providing more accurate evaluation results.

REFERENCES

[1] A. H. N. Sabet, Z. Zhao, and R. Gupta, "Subway: Minimizing data transfer during out-of-gpu-memory graph processing," in *Proceedings of the Fifteenth European Conference on Computer Systems*, 2020, pp. 1–16.

[2] Y. Zhang, D. Peng, X. Liao, H. Jin, H. Liu, L. Gu, and B. He, "Large-graph: An efficient dependency-aware gpu-accelerated large-scale graph processing," *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 18, no. 4, pp. 1–24, 2021.

[3] Z. Fu, M. Personick, and B. Thompson, "Mapgraph: A high level api for fast development of high performance graph analytics on gpus," in *Proceedings of workshop on GRAph data management experiences and systems*, 2014, pp. 1–6.

[4] Y. Zhang, X. Liao, L. Gu, H. Jin, K. Hu, H. Liu, and B. He, "Asyngraph: Maximizing data parallelism for efficient iterative graph processing on gpus," *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 17, no. 4, pp. 1–21, 2020.

[5] S. K. S. Hari, T. Tsai, M. Stephenson, S. W. Keckler, and J. Emer, "Sassifi: An architecture-level fault injection tool for gpu application resilience evaluation," in *2017 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2017, pp. 249–258.

[6] D. Sartzetakis, G. Papadimitriou, and D. Gizopoulos, "gpufi-4: A microarchitecture-level framework for assessing the cross-layer resilience of nvidia gpus," in *2022 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2022, pp. 35–45.

[7] B. Nicolae, A. Moody, E. Gonsiorowski, K. Mohror, and F. Cappello, "Veloc: Towards high performance adaptive asynchronous checkpointing at large scale," in *2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2019, pp. 911–920.

[8] P. H. Hochschild, P. Turner, J. C. Mogul, R. Govindaraju, P. Ranganathan, D. E. Culler, and A. Vahdat, "Cores that don't count," in *Proceedings of the Workshop on Hot Topics in Operating Systems*, ser. HotOS '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 9–16. [Online]. Available: https://doi.org/10.1145/3458336.3465297

[9] Q. Lu, G. Li, K. Pattabiraman, M. S. Gupta, and J. A. Rivers, "Configurable detection of sdc-causing errors in programs," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 16, no. 3, pp. 1–25, 2017.

[10] T. Tsai, S. K. S. Hari, M. Sullivan, O. Villa, and S. W. Keckler, "Nvbitfi: Dynamic fault injection for gpus," in *2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2021, pp. 284–291.

[11] Q. Lu, M. Farahani, J. Wei, A. Thomas, and K. Pattabiraman, "Llfi: An intermediate code-level fault injection tool for hardware faults," in *2015 IEEE International Conference on Software Quality, Reliability and Security*. IEEE, 2015, pp. 11–16.

[12] G. Li, K. Pattabiraman, and N. DeBardeleben, "Tensorfi: A configurable fault injector for tensorflow applications," in *2018 IEEE International symposium on software reliability engineering workshops (ISSREW)*. IEEE, 2018, pp. 313–320.

[13] A. R. Anwer, G. Li, K. Pattabiraman, M. Sullivan, T. Tsai, and S. K. S. Hari, "Gpu-trident: Efficient modeling of error propagation in gpu programs," in *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2020, pp. 1–15.

[14] B. Nie, L. Yang, A. Jog, and E. Smirni, "Fault site pruning for practical reliability analysis of gpgpu applications," in *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2018, pp. 749–761.

[15] S. K. S. Hari, S. V. Adve, H. Naeimi, and P. Ramachandran, "Relyzer: Exploiting application-level fault equivalence to analyze application resiliency to transient faults," *ACM SIGARCH Computer Architecture News*, vol. 40, no. 1, pp. 123–134, 2012.

[16] Z. Chen, G. Li, K. Pattabiraman, and N. DeBardeleben, "Binfi: An efficient fault injector for safety-critical machine learning systems," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '19. New York, NY, USA: Association for Computing Machinery, 2019. [Online]. Available: https://doi.org/10.1145/3295500.3356177

[17] Z. Li, H. Menon, K. Mohror, P.-T. Bremer, Y. Livant, and V. Pascucci, "Understanding a program's resiliency through error propagation," in *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 2021, pp. 362–373.

[18] O. Pusz, C. Dietrich, and D. Lohmann, "Data-flow-sensitive fault-space pruning for the injection of transient hardware faults," in *Proceedings of the 22nd ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems*, 2021, pp. 97–109.

[19] F. Khorasani, K. Vora, R. Gupta, and L. N. Bhuyan, "Cusha: vertex-centric graph processing on gpus," in *Proceedings of the 23rd international symposium on High-performance parallel and distributed computing*, 2014, pp. 239–252.

[20] A. H. Nodehi Sabet, J. Qiu, and Z. Zhao, "Tigr: Transforming irregular graphs for gpu-friendly graph processing," *ACM SIGPLAN Notices*, vol. 53, no. 2, pp. 622–636, 2018.

[21] A. Roy, I. Mihailovic, and W. Zwaenepoel, "X-stream: Edge-centric graph processing using streaming partitions," in *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, 2013, pp. 472–488.

[22] S. Zhou, R. Kannan, V. K. Prasanna, G. Seetharaman, and Q. Wu, "Hitgraph: High-throughput graph processing framework on fpga," *IEEE Transactions on Parallel and Distributed Systems*, vol. 30, no. 10, pp. 2249–2264, 2019.

[23] W. Han, D. Mawhirter, B. Wu, and M. Buland, "Graphie: Large-scale asynchronous graph traversals on just a gpu," in *2017 26th International Conference on Parallel Architectures and Compilation Techniques (PACT)*. IEEE, 2017, pp. 233–245.

[24] T. Ben-Nun, M. Sutton, S. Pai, and K. Pingali, "Groute: Asynchronous multi-gpu programming model with applications to large-scale graph processing," *ACM Transactions on Parallel Computing (TOPC)*, vol. 7, no. 3, pp. 1–27, 2020.

[25] B. Fang, Q. Guan, N. Debardeleben, K. Pattabiraman, and M. Ripeanu, "Letgo: A lightweight continuous framework for hpc applications under failures," in *Proceedings of the 26th International Symposium on High-Performance Parallel and Distributed Computing*, 2017, pp. 117–130.

[26] X. Wei, N. Jiang, H. Yue, X. Wang, J. Zhao, G. Li, and M. Qiu, "Approxdup: Developing an approximate instruction duplication mechanism for efficient sdc detection in gpgpus," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 4, pp. 1051–1064, 2024.

[27] A. Mahmoud, S. K. S. Hari, M. B. Sullivan, T. Tsai, and S. W. Keckler, "Optimizing software-directed instruction replication for gpu error detection," in *SC18: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2018, pp. 842–854.

[28] H. Yue, X. Wei, G. Li, J. Zhao, N. Jiang, and J. Tan, "G-sepm: Building an accurate and efficient soft error prediction model for gpgpus," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '21. New York, NY, USA: Association for Computing Machinery, 2021. [Online]. Available: https://doi.org/10.1145/3458817.3476170

[29] A. Bakhoda, G. L. Yuan, W. W. Fung, H. Wong, and T. M. Aamodt, "Analyzing cuda workloads using a detailed gpu simulator," in *2009 IEEE international symposium on performance analysis of systems and software*. IEEE, 2009, pp. 163–174.

[30] "Foldoc," http://vlado.fmf.uni-lj.si/pub/networks/data/dic/foldoc, 2024.

[31] "Grid plant, fb pages, and open flights," https://networkrepository.com/, 2024.

[32] L. Song, Y. Zhuo, X. Qian, H. Li, and Y. Chen, "Graphr: Accelerating graph processing using reram," in *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2018, pp. 531–543.

[33] K. Vora, C. Tian, R. Gupta, and Z. Hu, "Coral: Confined recovery in distributed asynchronous graph processing," in *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 223–236. [Online]. Available: https://doi.org/10.1145/3037697.3037747

[34] R. Venkatagiri, A. Mahmoud, S. K. S. Hari, and S. V. Adve, "Approxilyzer: Towards a systematic framework for instruction-level approximate computing and its application to hardware resiliency," in *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2016, pp. 1–14.

[35] B. Nie, A. Jog, and E. Smirni, "Characterizing accuracy-aware resilience of gpgpu applications," in *2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID)*, 2020, pp. 111–120.